JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class PipedInputStream

java.lang.Object
    java.io.InputStream
        java.io.PipedInputStream

All Implemented Interfaces:

Closeable, AutoCloseable

```
public class PipedInputStream
extends InputStream
```

A piped input stream should be connected to a piped output stream; the piped input stream then provides whatever data bytes are written to the piped output stream. Typically, data is read from a `PipedInputStream` object by one thread and data is written to the corresponding `PipedOutputStream` by some other thread. Attempting to use both objects from a single thread is not recommended, as it may deadlock the thread. The piped input stream contains a buffer, decoupling read operations from write operations, within limits. A pipe is said to be *broken* if a thread that was providing data bytes to the connected piped output stream is no longer alive.

Since:

1.0

See Also:

PipedOutputStream

Field Summary

Fields

Modifier and Type	Field	Description
protected byte[]	**buffer**	The circular buffer into which incoming data is placed.
protected int	**in**	The index of the position in the circular buffer at which the next byte of data will be stored when received from the connected piped output stream.
protected int	**out**	The index of the position in the circular buffer at

		which the next byte of data will be read by this piped input stream.
protected static final int	PIPE_SIZE	The default size of the pipe's circular input buffer.

Constructor Summary

Constructors

Constructor	Description
PipedInputStream()	Creates a PipedInputStream so that it is not yet connected.
PipedInputStream(int pipeSize)	Creates a PipedInputStream so that it is not yet connected and uses the specified pipe size for the pipe's buffer.
PipedInputStream (PipedOutputStream src)	Creates a PipedInputStream so that it is connected to the piped output stream src.
PipedInputStream (PipedOutputStream src, int pipeSize)	Creates a PipedInputStream so that it is connected to the piped output stream src and uses the specified pipe size for the pipe's buffer.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
int	available()	Returns the number of bytes that can be read from this input stream without blocking.
void	close()	Closes this input stream and releases any system

		resources associated with the stream.
void	**connect**(PipedOutputStream src)	Causes this piped input stream to be connected to the piped output stream src.
int	**read**()	Reads the next byte of data from this piped input stream.
int	**read**(byte[] b, int off, int len)	Reads up to len bytes of data from this piped input stream into an array of bytes.
protected void	**receive**(int b)	Receives a byte of data.

Methods declared in class InputStream

mark, markSupported, nullInputStream, read, readAllBytes, readNBytes, readNBytes, reset, skip, skipNBytes, transferTo

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

PIPE_SIZE

```
protected static final int PIPE_SIZE
```

The default size of the pipe's circular input buffer.

Since:

1.1

See Also:

Constant Field Values

buffer

```
protected byte[] buffer
```

The circular buffer into which incoming data is placed.

Since:

1.1

in

```
protected int in
```

The index of the position in the circular buffer at which the next byte of data will be stored when received from the connected piped output stream. `in < 0` implies the buffer is empty, `in == out` implies the buffer is full

Since:

1.1

out

```
protected int out
```

The index of the position in the circular buffer at which the next byte of data will be read by this piped input stream.

Since:

1.1

Constructor Details

PipedInputStream

```
public PipedInputStream(PipedOutputStream src)
                 throws IOException
```

Creates a `PipedInputStream` so that it is connected to the piped output stream `src`. Data bytes written to `src` will then be available as input from this stream.

Parameters:

`src` - the stream to connect to.

Throws:

`IOException` - if an I/O error occurs.

PipedInputStream

```
public PipedInputStream(PipedOutputStream src,
                        int pipeSize)
                 throws IOException
```

Creates a `PipedInputStream` so that it is connected to the piped output stream `src` and uses the specified pipe size for the pipe's buffer. Data bytes written to `src` will then be available as input from this stream.

Parameters:

`src` - the stream to connect to.

`pipeSize` - the size of the pipe's buffer.

Throws:

`IOException` - if an I/O error occurs.

`IllegalArgumentException` - if `pipeSize <= 0`.

Since:

1.6

PipedInputStream

```
public PipedInputStream()
```

Creates a `PipedInputStream` so that it is not yet connected. It must be connected to a `PipedOutputStream` before being used.

PipedInputStream

```
public PipedInputStream(int pipeSize)
```

Creates a `PipedInputStream` so that it is not yet connected and uses the specified pipe size for the pipe's buffer. It must be connected to a `PipedOutputStream` before being used.

Parameters:

`pipeSize` - the size of the pipe's buffer.

Throws:

`IllegalArgumentException` - if `pipeSize <= 0`.

Since:

1.6

Method Details

connect

```
public void connect(PipedOutputStream src)
             throws IOException
```

Causes this piped input stream to be connected to the piped output stream `src`. If this object is already connected to some other piped output stream, an `IOException` is thrown.

If `src` is an unconnected piped output stream and `snk` is an unconnected piped input stream, they may be connected by either the call:

```
    snk.connect(src)
```

or the call:

```
    src.connect(snk)
```

The two calls have the same effect.

Parameters:

`src` - The piped output stream to connect to.

Throws:

`IOException` - if an I/O error occurs.

receive

```
protected void receive(int b)
                throws IOException
```

Receives a byte of data. This method will block if no input is available.

Parameters:

`b` - the byte being received

Throws:

`IOException` - If the pipe is `broken`, `unconnected`, closed, or if an I/O error occurs.

Since:

1.1

read

```
public int read()
         throws IOException
```

Reads the next byte of data from this piped input stream. The value byte is returned as an `int` in the range `0` to `255`. This method blocks until input data is available, the end of the stream is detected, or an exception is thrown.

Specified by:

`read` in class `InputStream`

Returns:

the next byte of data, or `-1` if the end of the stream is reached.

Throws:

`IOException` - if the pipe is `unconnected`, `broken`, closed, or if an I/O error occurs.

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads up to `len` bytes of data from this piped input stream into an array of bytes. Less than `len` bytes will be read if the end of the data stream is reached or if `len` exceeds the pipe's buffer size. If `len` is zero, then no bytes are read and 0 is returned; otherwise, the method blocks until at least 1 byte of input is available, end of the stream has been detected, or an exception is thrown.

Overrides:

`read` in class `InputStream`

Parameters:

`b` - the buffer into which the data is read.

`off` - the start offset in array `b` at which the data is written.

`len` - the maximum number of bytes to read.

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

Throws:

`NullPointerException` - If `b` is `null`.

IndexOutOfBoundsException - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

IOException - if the pipe is broken, unconnected, closed, or if an I/O error occurs.

See Also:

InputStream.read()

available

```
public int available()
              throws IOException
```

Returns the number of bytes that can be read from this input stream without blocking.

Overrides:

available in class InputStream

Returns:

the number of bytes that can be read from this input stream without blocking, or `0` if this input stream has been closed by invoking its close() method, or if the pipe is unconnected, or broken.

Throws:

IOException - if an I/O error occurs.

Since:

1.0.2

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream.

Specified by:

close in interface AutoCloseable

Specified by:

close in interface Closeable

Overrides:

close in class InputStream

Throws:

IOException - if an I/O error occurs.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.